Equipo 03

Nombre: María José Cedillo Mata **Matrícula**: 610256
Nombre: Alejandra Morón Rdz. **Matrícula**: 587823
Nombre: Fernando Olivares Del Valle **Matrícula**: 600046
Nombre: Ricardo Arath Sánchez Aguirre **Matrícula**: 583928

Módulo II - Actividad de investigación y práctica

Institución educativa: Universidad de Monterrey

Materia: Sistemas Operativos

Profesor: Raúl Morales Salcedo

Gpo: 3305 8:30 am **Semestre:** 6to

Fecha: 7 de mayo de 2026

"Damos nuestra palabra de que hemos realizado esta actividad con integridad académica"

Índice

Introducción..... 3
Objetivo del Sistema..... 3
Marco Teórico / Investigación Conceptual..... 3
Arquitectura del Sistema..... 10
Diseño concurrente..... 10
Simulación de cores..... 10
Implementación de algoritmos..... 10
Manejo de concurrencia..... 10
Resultados experimentales..... 10
Escenarios de prueba y evidencias..... 10
Análisis comparativo..... 10
Conclusiones..... 10
Trabajo futuro..... 10
Referencias..... 10

Introducción

El proyecto es un simulador interactivo de sistemas operativos que permite visualizar y analizar la gestión de recursos en tiempo real, generando métricas en tiempo real y gráficas comparativas que facilitan el estudio del comportamiento del kernel bajo diversas cargas de trabajo. El alcance incluye los módulos de procesos, estados, scheduling, paginación, reemplazo de páginas, threads y un derivado de threads llamado multicore con más procesos.

Objetivo del Sistema

El objetivo del proyecto es desarrollar un simulador visual que represente de manera gráfica el funcionamiento tanto de algoritmos de planificación de CPU como algoritmos de paginación de memoria. Logramos visualizar como equipo cómo cambian los estados de procesos y cómo se administra la memoria.

Marco Teórico / Investigación Conceptual

1. Procesos

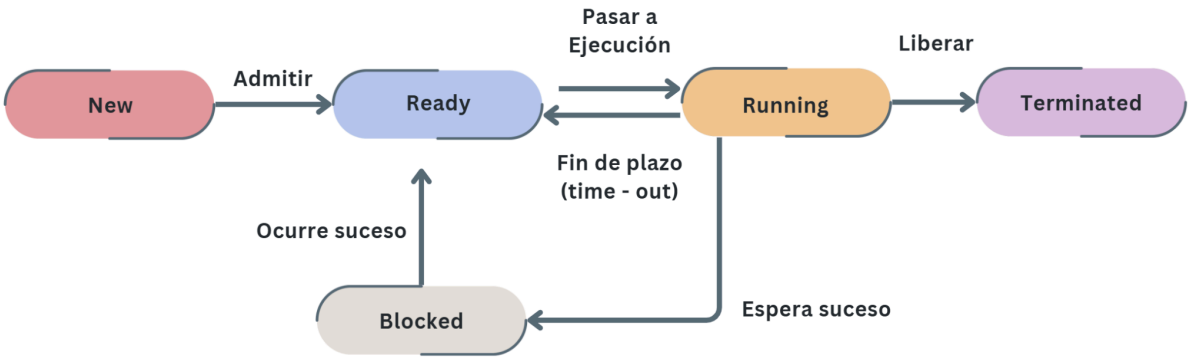
Los procesos son los programas o instrucciones que se ejecutan en el sistema operativo. Se ejecutan independientemente y hace uso de recursos como la memoria, procesador y dispositivos input/output del sistema. (Martínez J. 2024)

Los estados de un proceso son los siguientes:

1. New. Procesos que acaban de ser definidos pero que aún no son admitidos por el sistema operativos como ejecutables. Una vez admitido el proceso pasa al estado Running.
2. Ready. Procesos que cuentan con todos los recursos necesarios para su ejecución y esperan a se les del control del procesador.
3. Running. Tienen control del procesador y se mantiene ejecutado.
4. Blocked. Procesos que carecen de algún recurso necesario para su ejecución. Espera un evento antes de volver al estado Ready.
5. Terminated. Procesos excluidos por el sistema operativo del grupo de ejecutables. Ocurre cuando el proceso termina normalmente, se abandona por algún error o cuando otro proceso con mayor autoridad lo termina.

(Buzón M., 2025)

Diagrama 2. Modelo de Estado de Procesos



(Fuente: Elaboración Propia)-

El diagrama muestra los diferentes estados por los que puede pasar un proceso dentro de un sistema operativo: New, Ready, Running, Blocked y Terminated. Se ilustran las transiciones entre estos estados, por ejemplo, cómo un proceso pasa de Ready a Running cuando el CPU lo ejecuta, o de Running a Blocked cuando necesita un recurso. Este modelo es fundamental para entender cómo el sistema operativo administra múltiples procesos y cómo decide cuál ejecutar en cada momento.

Hay 2 tipos de procesos, los de usuario y los en segundo plano. Los procesos de usuario se ejecutan en nombre de un usuario con los permisos de este. Por ejemplo bash o chrome. Por otro lado, los procesos en segundo plano no se muestran en la pantalla de usuario y no requieren de interacción. Algunos son ssh y cron. (Martínez J. 2024)

La administración, coordinación y control de procesos se lleva a cabo mediante el uso de algoritmos de planificación de procesos. Los cuales determinan qué proceso se ejecuta en un momento dado y cómo se gestionan recursos del sistema entre ellos. Cada proceso cuenta con información sobre su estado, identificador, prioridad, y recursos asignados, de modo que el planificador toma decisiones sobre el proceso a ejecutar a continuación. (TecnoDigital. 2025)

2. Threads

Un hilo es una unidad de ejecución que se ejecuta dentro de un proceso. Varios hilos pueden ejecutarse simultáneamente dentro del mismo proceso y compartir los mismos recursos, lo cual permite una gestión eficiente de ellos y un procesamiento de mayor velocidad. (Limited, H. G. 2025)

Algunas de sus características son:

- Ligereza. Son más ligeros que los procesos, por lo que crear o destruirlos consume menos recursos del sistema.
- Comparte Recursos. Los hilos de un mismo proceso comparten el mismo espacio de memoria, haciendo la comunicación entre ellos más eficiente.
- Concurrencia. Los hilos permiten que múltiples tareas se ejecuten al mismo tiempo, mejorando la capacidad de respuesta.

(Cortés D., 2024)

Tabla 1. Cuadro comparativo entre procesos e hilos

Característica	**Proceso**	**Hilo**
Definición	Unidad de ejecución independiente.	Unidad de ejecución que se ejecuta dentro de un proceso.
Espacio de dirección	Espacio de direcciones privado y propio.	Comparte el espacio de direcciones con el proceso.
Uso de recursos	Consume más recursos.	Consume menos recursos.
Contacto	Complejo y lento (IPC)	Rápido y fácil (memoria compartida)

(Limited, H. G. 2025)

En la Tabla 1 se puede observar que los hilos son recomendables cuando se quiere una mejor gestión de memoria o bien, un uso menor de esta y de recursos del sistema.

3. Fork

Se refiere a una llamada al sistema que crea una copia exacta de un proceso existente. El proceso original se le llama padre y a la copia hijo. Se ejecutan de manera independiente, pero comparten el mismo código, datos, memoria y estado.

Fork es utilizado para dividir tareas en varios procesos paralelos, de manera que cada uno se ejecute en un núcleo de la CPU diferente. Esto aumenta la eficiencia computacional al abordar diferentes partes de un problema de manera simultánea.

La llamada fork regresa dos veces cuando tiene éxito. Una en el proceso padre y otra en el hijo, devolviendo el PID del proceso hijo en el padre y el valor 0 en el hijo. Se toma en cuenta que al especificar las condiciones de un bucle es posible distinguir diferentes rutas de código para procesos.

Además, después de ejecutar fork(), ambos procesos continúan desde la misma instrucción donde se realizó la llamada, lo que permite que el flujo del programa se bifurque de forma controlada. Aunque inicialmente comparten el mismo contenido en memoria, el sistema operativo aplica técnicas como *copy-on-write* para evitar duplicaciones innecesarias hasta que alguno de los procesos modifica datos. Esto hace que la creación de procesos sea más eficiente y permite implementar estructuras como servidores concurrentes o ejecución de múltiples tareas sin bloquear el proceso principal. (Hu J., 2023)

4. Scheduling

Scheduling o planificación de procesos tiene como objetivo principal determinar el orden en el que los procesos acceden a la CPU. Al gestionar los recursos, es posible garantizar el funcionamiento eficiente del sistema. Cuando hay múltiples procesos compitiendo por la CPU, la planificación ayuda a maximizar el rendimiento, minimizar el tiempo de espera y evitar la inanición.

Los algoritmos de planificación determinan cómo se asigna la CPU a los procesos en la cola de preparados. Se dividen en preventivos y no preventivos.

La programación preventiva es un método de programación en el que la mayoría de las tareas se asignan con sus prioridades. Si es importante ejecutar una tarea con mayor prioridad antes que otra tarea de menor prioridad, el proceso que mantiene ocupada la CPU liberará la CPU ya sea cambiando de contexto o finalizando.

En la programación no preventiva, una vez que el procesador inicia su ejecución, debe finalizar antes de ejecutar el otro. No se puede detener a la mitad.

(Brooks N., 2024)

Algunos de los algoritmos de planificación son:

1. First-Come, First-Served (FCFS). Los procesos se ejecutan en el orden en que llegan a la cola de preparados. No preventivo.
    - Ventajas: Fácil de implementar y justo en términos de orden de llegada.
    - Desventajas: Puede llevar al problema del convoy effect, donde un proceso largo puede bloquear a otros procesos más cortos, aumentando el tiempo de espera promedio.
2. Shortest Job Next (SJN) o Shortest Job First (SJF): El proceso con el tiempo de ejecución más corto es el que se ejecuta primero. No preventivo.
    - Ventajas: Minimiza el tiempo de espera promedio.
    - Desventajas: Además, puede llevar a la inanición de procesos más largos..
3. Round-Robin (RR). Los procesos se ejecutan en un orden circular, con un tiempo de ejecución fijo llamado quantum. Preventivos
    - Ventajas: Garantiza equidad, ya que todos los procesos reciben un tiempo de CPU igual.
    - Desventajas: Si el quantum es demasiado corto, el overhead de puede ser significativo y si es demasiado largo funcionará como FCFS.
4. Priority Scheduling. Cada proceso tiene una prioridad asignada, y la CPU se asigna al proceso con la prioridad más alta. Preventivos
    - Ventajas: Permite un control granular sobre la asignación de CPU según la importancia de los procesos.
    - Desventajas: Puede llevar a la inanición de procesos de baja prioridad.
5. Multilevel Queue Scheduling. Los procesos se asignan a diferentes colas según ciertas características y cada cola tiene su propio algoritmo de planificación. Preventivos

- Ventajas: Flexibilidad y control.
- Desventajas: La asignación rígida a colas puede llevar a un mal uso de los recursos si una cola está vacía mientras que otra está sobrecargada.

6. Multilevel Feedback Queue. Similar a la multilevel queue, pero permite que los procesos cambien de cola según su comportamiento y tiempo de ejecución. Preventivos
    - Ventajas: Combina lo mejor de varios algoritmos, adaptándose dinámicamente al comportamiento del proceso.
    - Desventajas: Es complejo de implementar y afinar correctamente.

Las políticas de planificación definen las reglas y objetivos específicos que el sistema operativo sigue al decidir cómo asignar la CPU a los procesos. Estas políticas se evalúan según varios criterios clave:

- Throughput. Mide el número de procesos que se completan en un período de tiempo determinado. Un alto throughput indica que el sistema está utilizando la CPU de manera eficiente.
- Tiempo de Respuesta. Es el tiempo que tarda un proceso desde que se envía hasta que comienza su ejecución.
- Tiempo de Espera. Calcula el tiempo total que un proceso pasa en la cola de preparados. Se busca minimizar el promedio para mejorar la eficiencia global del sistema.
- Tiempo de Turnaround. Es el tiempo total que tarda un proceso en completarse, desde su llegada hasta su terminación. Minimizar este tiempo aumenta la productividad del sistema.

(Ti D. 2024)

5. Memoria

La memoria virtual es un mecanismo de almacenamiento que ofrece la ilusión de una memoria principal grande, de modo que se trata una parte de la memoria secundaria como principal. En la memoria virtual, el usuario puede almacenar

procesos con un tamaño mayor que la memoria principal disponible. El sistema operativo carga distintas partes de más de un proceso en la memoria principal. (Brooks N. 2024)

La gestión de memoria paginada se basa en la división de la memoria principal de la cpu en unidades de tamaño fijo conocidas como marcos de página. Esta unidad de gestión de memoria de hardware asigna páginas a marcos que deben asignarse por páginas. (Brooks N. 2024)

La gestión de memoria segmentada es un método que no proporciona al programa un espacio de direcciones lineal y contigua. Los segmentos necesitan soporte de hardware en forma de tabla de segmentos, Contiene la dirección física de la sección en la memoria, el tamaño y otros datos como bits de protección de acceso y estado (Brooks N. 2024).

La paginación con segmentación combina ambas técnicas. El programa se divide en segmentos y cada uno se divide a su vez en páginas. Esto permite que los segmentos de diferentes tamaños se gestionen de manera más eficiente, al tiempo que se aprovechan las ventajas de paginación, cómo la reducción de la fragmentación interna y la mejora en la utilización de memoria. (Instituto Consorcio Clavijero, s.f.)

Un tamaño de página adecuado puede mejorar la eficiencia del sistema operativo al minimizar la fragmentación interna y externa, lo que a su vez reduce el tiempo de acceso y mejora el rendimiento general del sistema. Una página grande puede causar uso insuficiente de la memoria al no ocupar completamente los marcos de la página, mientras que un tamaño pequeño puede resultar en un uso excesivo de memoria al ocupar los marcos con páginas de diferentes procesos. (Cruzito R. 2025)

Arquitectura del Sistema

La arquitectura del simulador está diseñada bajo un enfoque frontend-only del lado del usuario, donde toda la lógica de ejecución, visualización y simulación se realiza en el navegador mediante JavaScript. No existe backend ni almacenamiento persistente, por lo que el sistema opera como una aplicación autónoma. La estructura general se basa en la separación por módulos funcionales, donde cada archivo HTML representa una “pestaña” o vista independiente que simula un componente específico de un sistema operativo.

El sistema implementa una arquitectura modular por vistas, en la cual cada módulo encapsula su propia lógica, estado y representación visual. Cada pestaña sigue implícitamente una separación entre datos (modelo), lógica de simulación (controlador) y representación visual (vista). El estado se maneja mediante variables

globales dentro de cada archivo, lo que permite manipular directamente procesos, colas o estructuras de memoria.

El sistema está compuesto por las siguientes pestañas principales, cada una con un propósito específico dentro de la simulación:

- El archivo index.html corresponde a la simulación de los estados de los procesos y representa el ciclo de vida completo de un proceso dentro del sistema operativo. En esta vista se modelan explícitamente los estados clásicos: *New, Ready, Running, Waiting y Terminated*. La interfaz permite observar cómo los procesos se crean, ingresan a la cola de listos, son asignados al CPU, pueden bloquearse (por ejemplo, simulando operaciones de entrada/salida) y finalmente terminan su ejecución. Esta pestaña simula la dinámica básica de gestión de procesos en un sistema operativo, mostrando visualmente las transiciones de estado y cómo el scheduler interactúa con dichas transiciones. Es una representación directa del modelo de estados de procesos.

- El módulo scheduling.html está dedicado a la simulación de algoritmos de planificación de CPU. En esta pestaña, el sistema permite definir procesos con atributos como tiempo de llegada, tiempo de ráfaga o prioridad. A partir de estos datos, se ejecutan distintos algoritmos de planificación. Los algoritmos non-preemptive son FCFS, SJF y HRRN, por otro lado, los algoritmos preemptive son Round Robin, SRTF, Priority (Preemptive), Multilevel Queue, Multilevel Feedback Queue. El módulo muestra cómo los procesos son seleccionados para ejecución mediante diagramas Gantt y representaciones temporales, permitiendo observar métricas como tiempos de espera y turnaround. Esta pestaña simula el comportamiento del scheduler del sistema operativo y cómo toma decisiones para asignar el CPU.

- El archivo threads.html simula la ejecución de múltiples hilos dentro de un mismo proceso. En esta vista, cada proceso puede contener varios hilos que se ejecutan de manera aparentemente concurrente. El sistema representa los hilos como subunidades de ejecución que comparten recursos del proceso padre, mostrando cómo pueden avanzar de manera independiente dentro del mismo contexto. Esta pestaña modela el concepto de multithreading, enfatizando la diferencia entre procesos y hilos, así como la capacidad de un proceso para realizar múltiples tareas simultáneamente de forma conceptual.

- El módulo multicore.html está enfocado en la simulación de sistemas con múltiples núcleos de procesamiento. En esta pestaña, el sistema representa varios cores de CPU, cada uno capaz de ejecutar un proceso de manera independiente. Los procesos son distribuidos entre los distintos núcleos, y la interfaz muestra cómo cada core ejecuta su carga de trabajo en paralelo con los demás. Esta vista simula arquitecturas de multiprocesamiento simétrico, permitiendo observar conceptos como balanceo de carga y asignación de procesos a múltiples unidades de ejecución.

- El archivo paginacion.html implementa la simulación de memoria virtual mediante paginación. En esta pestaña, la memoria se divide en marcos (frames) y los procesos en páginas. El sistema muestra cómo las páginas de un proceso se cargan en memoria física y cómo se realiza el mapeo entre direcciones virtuales y físicas mediante una tabla de páginas. La interfaz permite visualizar el estado de la memoria en tiempo real, identificando qué páginas están cargadas y cuáles no. Esta pestaña simula el funcionamiento de la administración de memoria en sistemas operativos modernos.

- El módulo replacement.html se enfoca en los algoritmos de reemplazo de páginas. En esta vista, el sistema recibe una secuencia de referencias de memoria y simula cómo se cargan las páginas en memoria física limitada. Cuando la memoria se llena, se aplican algoritmos como FIFO, LRU o el óptimo para decidir qué página debe ser reemplazada. La interfaz muestra paso a paso el estado de la memoria, permitiendo observar los fallos de página (page faults) y el comportamiento de cada algoritmo. Esta pestaña simula uno de los problemas fundamentales de la administración de memoria: la sustitución eficiente de páginas.

La arquitectura del sistema permite abordar de manera modular los distintos componentes de un sistema operativo. Cada pestaña funciona como una simulación independiente, pero todas comparten un enfoque común basado en la representación visual, el uso de estructuras de datos simples y la simulación del tiempo mediante eventos discretos. Esta organización facilita tanto el desarrollo como la comprensión del sistema, ya que cada módulo se enfoca en un concepto específico sin interferir directamente con los demás.

Diseño concurrente

El diseño de concurrencia del sistema se fundamenta en la necesidad de representar la ejecución simultánea de múltiples procesos dentro de las limitaciones de un entorno de ejecución monohilo como lo es JavaScript en el navegador. En este contexto, la

concurrencia se modela a través de una abstracción basada en eventos discretos, donde el tiempo del sistema avanza en intervalos definidos y las entidades de ejecución son actualizadas de forma secuencial.

El sistema adopta un modelo de concurrencia basado en intercalado de operaciones (interleaving), en el cual múltiples procesos progresan aparentemente al mismo tiempo mediante la alternancia de su ejecución en pequeños intervalos de tiempo. Este enfoque es característico de sistemas de tiempo compartido, donde el procesador se asigna dinámicamente a diferentes procesos con el objetivo de maximizar la utilización del CPU y mejorar la capacidad de respuesta del sistema.

Para soportar este modelo, el diseño incorpora una representación explícita de la máquina de estados de los procesos, donde cada proceso puede encontrarse en estados como nuevo, listo, en ejecución, en espera o terminado. Estas entidades son modeladas como estructuras de datos que encapsulan atributos relevantes, tales como tiempos de llegada, ráfaga, tiempo restante y estado actual. La existencia de estos estados permite definir reglas claras de transición, las cuales son fundamentales para simular el comportamiento concurrente.

Adicionalmente, el diseño contempla la utilización de colas de planificación, particularmente la cola de procesos listos (ready queue), la cual almacena los procesos que están en condiciones de ser ejecutados. Estas colas son estructuras esenciales en la organización de la concurrencia, ya que determinan el orden en el cual los procesos acceden al CPU. Dependiendo del algoritmo de planificación seleccionado, el orden de estas colas puede variar dinámicamente.

Otro elemento clave del diseño es la abstracción del reloj del sistema, el cual actúa como el eje temporal de toda la simulación. Este reloj define los puntos discretos en los cuales el estado del sistema es evaluado y actualizado. De esta manera, el diseño establece una secuencia lógica de eventos que permite coordinar la evolución de múltiples procesos de forma consistente.

En conjunto, el diseño de concurrencia del simulador proporciona una representación conceptual de los sistemas multiprogramados, donde múltiples procesos coexisten en memoria y compiten por recursos compartidos. El modelo logra capturar los principios fundamentales de la concurrencia, como la compartición de recursos, la planificación y la evolución temporal de procesos.

Simulación de cores

La simulación de múltiples núcleos de procesamiento constituye uno de los componentes más relevantes del sistema, ya que permite representar de manera conceptual el comportamiento de arquitecturas modernas de cómputo basadas en paralelismo. En este simulador, el modelo multicore se implementa mediante la abstracción de cada núcleo como una entidad lógica independiente, la cual posee su propio estado de ejecución, un proceso activo asignado y, en algunos casos, una estructura de cola local o compartida desde la cual obtiene nuevas tareas.

Cada núcleo es representado internamente mediante una estructura de datos que encapsula atributos como identificador del core, proceso actualmente en ejecución, tiempo restante del proceso y estado operativo. A diferencia de una implementación real a nivel de hardware, donde cada núcleo ejecuta instrucciones de manera verdaderamente paralela, en este sistema todos los núcleos son simulados dentro de un único hilo de ejecución, lo que implica que su comportamiento debe modelarse mediante iteraciones secuenciales cuidadosamente estructuradas.

El mecanismo de simulación se fundamenta en un ciclo de reloj global, el cual es gestionado mediante funciones temporizadas del entorno JavaScript, como setInterval. En cada “tick” de este reloj, el sistema recorre secuencialmente la colección de núcleos y ejecuta una serie de operaciones deterministas sobre cada uno de ellos. Estas operaciones incluyen la actualización del proceso en ejecución, la reducción de su tiempo restante (simulando la ejecución de instrucciones), la verificación de finalización del proceso y la eventual asignación de un nuevo proceso en caso de que el anterior haya concluido.

La asignación de procesos a núcleos puede implementarse mediante distintas estrategias simplificadas, como distribución equitativa inicial, asignación bajo demanda o extracción desde una cola global compartida. Este comportamiento permite modelar de manera abstracta el concepto de balanceo de carga, en el cual los procesos son distribuidos entre múltiples unidades de ejecución para optimizar el uso de los recursos del sistema. Aunque el simulador no implementa algoritmos avanzados de balanceo dinámico, sí reproduce la idea fundamental de que múltiples procesos pueden ser atendidos simultáneamente por diferentes núcleos.

De manera visual cada núcleo es representado como una unidad independiente dentro de la interfaz, mostrando en tiempo real el proceso que ejecuta, su progreso y su estado. Esta representación genera la percepción de paralelismo, ya que múltiples procesos parecen avanzar de manera simultánea.

Esta simulación corresponde a un modelo simplificado de multiprocesamiento simétrico (SMP), donde todos los núcleos comparten acceso a una misma memoria y pueden ejecutar procesos indistintamente. El simulador logra capturar la esencia del paralelismo a nivel de hardware y permite comprender cómo múltiples unidades de procesamiento pueden cooperar en la ejecución de tareas.

Implementación de algoritmos

5.2 Implementación de Algoritmos de Scheduling

5.2.1 Algoritmos No-Preemptive

FCFS (First-Come, First-Served)

Estructura de datos:

- Array processes[] ordenado por arrival time (at)
- Cola lógica: los procesos se atienden en orden de llegada

Pseudocódigo:

```
processes = procesos ordenados por arrival_time
t = 0
completed = 0

while (completed < total_procesos):
    available = procesos donde (at ≤ t AND rt > 0)

    if (available existe):
        seleccionado = available[0]  // primer proceso disponible
        tiempo_ejecución = seleccionado.rt
        t += tiempo_ejecución
        seleccionado.rt = 0
        seleccionado.completion_time = t
        completed++
    else:
        t++  // CPU idle
```

Implementación en código:

- Archivo: scheduling.html
- Criterio de selección: available.sort((a, b) => a.at - b.at)[0]
- Complejidad: $O(n^2)$ donde n = número de procesos

Ejemplo con datos reales:

Proceso	AT	BT	CT	TAT	WT
P1	0	5	5	5	0
P2	1	3	8	7	4
P3	2	1	9	7	6

P4	5	4	13	8	4
Promedio	—	—	—	6.75	3.5

SJF (Shortest Job First)

Estructura de datos:

- Array processes[] con atributo bt (burst time)
- Selección basada en burst time mínimo

Pseudocódigo:

```
mientras (existan procesos con rt > 0):
    available = procesos donde (at ≤ t AND rt > 0)

    if (available existe):
        seleccionado = MIN(available, by burst_time)
        t += seleccionado.rt
        seleccionado.rt = 0
        seleccionado.ct = t
    else:
        t++  // idle
```

Implementación en código:

- Archivo: scheduling.html
- Criterio: available.sort((a, b) => a.bt - b.bt || a.at - b.at)[0]
- Ventaja implementada: Minimiza tiempo de espera promedio

Complejidad: $O(n^2 \log n)$ por ordenamiento en cada ciclo

Características especiales:

- Óptimo para minimizar tiempo de espera promedio
- Desventaja: Puede causar inanición (starvation) de procesos largos

---

PRIORITY

Estructura de datos:

- Array processes[] con atributo prio (prioridad numérica)
- Prioridad: números menores = mayor prioridad

Pseudocódigo:

```
mientras (proceso con rt > 0):
    available = procesos donde (at ≤ t AND rt > 0)

    if (available existe):
        seleccionado = MIN(available, by prio)
        t += seleccionado.rt
        seleccionado.rt = 0
        seleccionado.ct = t
    else:
        t++
```

Implementación:

- Criterio: available.sort((a, b) => a.prio - b.prio || a.at - b.at)[0]
- Atributo prio: configurable en GUI (rango: cualquier número)

Desventaja identificada:

- Posible starvation de procesos de baja prioridad

5.2.2 Algoritmos Preemptive

RR (Round Robin)

Parámetro crítico:

- Quantum: configurable en GUI (default = 2 unidades de tiempo)
- El quantum es variable durante la simulación

Estructura de datos:

```
let rrQueue = [];  // cola para Round Robin
let quantum = documento.getElementById("quantumInput").value;
```

Pseudocódigo:

```
rrQueue = []
mientras (existan procesos con rt > 0):
    // Encolamos nuevos procesos disponibles
    plist.filter(p => p.at ≤ t AND p.rt > 0 AND
!rrQueue.includes(p))
        .forEach(p => rrQueue.push(p))

    if (rrQueue.length > 0):
        proceso_actual = rrQueue.dequeue()

        // Ejecutar por el quantum o lo que quede
        tiempo_ejecución = min(proceso_actual.rt, quantum)
        t += tiempo_ejecución
        proceso_actual.rt -= tiempo_ejecución

        // Si aún tiene tiempo restante, reinsertar
        if (proceso_actual.rt > 0):
            rrQueue.enqueue(proceso_actual)
        else:
            proceso_actual.ct = t
    else:
        t++  // CPU idle
```

Implementación en código:

- Archivo: scheduling.html:500-530
- Línea visible en GUI: <input type="number" id="quantumInput" value="2" min="1">
- La cola se gestiona con .shift() y .push()

Quantum utilizado en simulaciones: 2 unidades de tiempo

Características:

- Equidad: Todos los procesos reciben tiempo de CPU
- Context Switches: Elevados con quantum pequeño
- Quantum óptimo: Depende de carga y características de procesos

Ejemplo de ejecución RR(q=2):

Tiempo	Proceso	RT antes	Ejecución	RT después	Acción
0-2	P1(5)	5	2	3	Ejecuta, reinserta

2-3	P2(3)	3	1	2	Ejecuta (solo 1, menos que quantum)
3-4	P3(1)	1	1	0	Ejecuta, termina
4-6	P1(3)	3	2	1	Ejecuta, reinserta

5.3 Implementación de Algoritmos de Page Replacement

5.3.1 FIFO (First-In, First-Out)

Estructura de datos:

```
let frames = [];  // Array que actúa como queue
```

Algoritmo:

```
para cada referencia de página p:
    if (p está en frames):
        // Hit
        hits++
    else:
        // Fault
        faults++
        if (frames.length < capacidad):
            frames.push(p)
        else:
            página_víctima = frames.shift()  // Saca el primero
            frames.push(p)                   // Inserta al final

    guardar_snapshot(frames)
```

Implementación en código:

- Función: simulateFIFO(refs, fc) línea ~520
- Complejidad: O(n) donde n = número de referencias

Ventajas:

- Simple de implementar
- Bajo overhead computacional

Desventajas:

- Sufre del anomalía de Belady (más frames → más faults en algunos casos)
- No considera patrones de acceso

5.3.2 LRU (Least Recently Used)

Estructura de datos:

```
let frames = [];      // Páginas en memoria
let timestamps = [];  // Orden de último acceso
```

Algoritmo:

```
para cada referencia de página p:
    idx = posición de p en frames

    if (idx !== -1):
        // Hit
        hits++
        frames.remove(idx)      // Sacar de su posición
        frames.push(p)          // Insertar al final (MRU)
    else:
        // Fault
        faults++
        if (frames.length < capacidad):
            frames.push(p)
        else:
            página_víctima = frames[0]  // Saca LRU (primero)
            frames.remove(0)
            frames.push(p)

    guardar_snapshot(frames)
```

Implementación en código:

- Función: simulateLRU(refs, fc) línea ~530
- Usa .indexOf() y .splice() para manipular array
- Complejidad: O(n) por referencia

Características:

- Casi óptimo en práctica
- Más caro computacionalmente que FIFO
- Rastrea uso reciente

5.3.3 OPTIMAL (Belady's Algorithm)

Estructura de datos:

```
let frames = [];
```

Algoritmo:

```
para cada referencia de página p en posición i:
    if (p está en frames):
        // Hit
        hits++
    else:
        // Fault
        faults++
        if (frames.length < capacidad):
            frames.push(p)
        else:
            // Buscar página que se usará más lejos en el futuro
            farthest = -1
            replaceIdx = 0

            para cada página en frames:
                próximo_acceso = buscar próxima ocurrencia en
refs[i+1:]
                if (próximo_acceso === -1):  // Nunca se usa más
                    replaceIdx = índice
                    break
                else if (próximo_acceso > farthest):
                    farthest = próximo_acceso
                    replaceIdx = índice

            página_víctima = frames[replaceIdx]
            frames[replaceIdx] = p

    guardar_snapshot(frames)
```

Implementación en código:

- Función: simulateOPT(refs, fc) línea ~535
- Búsqueda: refs.slice(i+1).indexOf(frames[j])
- Complejidad: O(n·m) donde m = tamaño de frames

Nota importante: Este es óptimo en teoría pero no implementable en sistemas reales porque requiere conocer el futuro. Se usa como benchmark de comparación.

5.3.4 CLOCK (Clock Algorithm with Reference Bit)

Estructura de datos:

```
let frames = new Array(fc).fill(null);
let bits = new Array(fc).fill(0);  // Bit de referencia
let ptr = 0;                       // Puntero circular
```

Algoritmo:

```
ptr = 0
para cada referencia de página p:
    fi = índice donde p está en frames

    if (p está en frames):
        // Hit
        hits++
        bits[fi] = 1  // Marcar como accedido recientemente
    else:
        // Fault
        faults++

        // Buscar víctima: página con bit=0
        mientras (true):
            if (bits[ptr] === 0):
                // Esta página tiene bit 0, reemplazarla
                página_víctima = frames[ptr]
                frames[ptr] = p
                bits[ptr] = 1
                ptr = (ptr + 1) % capacidad
                break
            else:
```

```
                // Dar segunda chance: poner bit a 0 y avanzar
                bits[ptr] = 0
                ptr = (ptr + 1) % capacidad

    guardar_snapshot(frames)
```

Implementación en código:

- Función: simulateCLOCK(refs, fc) línea ~545
- Usa aritmética modular: ptr = (ptr + 1) % fc
- Complejidad: O(n) amortizado

Características:

- Aproximación práctica a LRU
- Bajo overhead (solo requiere bit de referencia)
- Usado en sistemas operativos reales (Linux, Unix)

5.3.5 Second Chance (Variante mejorada de CLOCK)

Estructura de datos:

```
let frames = [];
let bits = [];  // Bit de segunda chance
```

Algoritmo:

```
para cada referencia de página p:
    idx = búscar p en frames

    if (idx !== -1):
        // Hit
        hits++
        bits[idx] = 1
    else:
        // Fault
        faults++
        if (frames.length < capacidad):
            frames.push(p)
            bits.push(1)
        else:
```

```
            // Buscar página sin segunda chance (bit=0)
            replaced = false

            mientras (!replaced):
                para cada página en frames:
                    if (bits[índice] === 0):
                        página_víctima = frames[índice]
                        frames[índice] = p
                        bits[índice] = 1
                        replaced = true
                        break
                    else:
                        // Dar segunda chance
                        bits[índice] = 0

    guardar_snapshot(frames)
```

Implementación en código:

- Función: simulateSC(refs, fc) línea ~560
- Usa arrays paralelos: frames[] y bits[]
- Complejidad: O(n) por referencia

Diferencias con CLOCK:

- No usa puntero circular
- Búsqueda lineal en lugar de circular
- Conceptualmente similar, implementación diferente

5.4 Estructura de Steps y Snapshots

Cada simulación genera un array steps[] que contiene:

```
{
  page: número_de_página,           // La página referenciada
  fault: boolean,                   // ¿Fue fault o hit?
  entered: número_de_página,        // Página que entra
  exited: número_de_página | null,  // Página que sale (null si no
hay evicción)
  framesSnapshot: [array],          // Estado de memoria después
  action: string                    // Descripción legible
```

```
}
```

Ejemplo de step generado:

```
{
  page: 7,
  fault: true,
  entered: 7,
  exited: 0,
  framesSnapshot: [7, 1, 2],
  action: "Replaced 0 with 7"
}
```

5.5 Métricas Calculadas en Tiempo Real

```
Para CPU Scheduling:
// Por proceso:
respTime = tiempo_de_inicio - arrival_time
tat = completion_time - arrival_time
wt = tat - burst_time

// Globales:
avg_tat = sum(tat) / num_procesos
avg_wt = sum(wt) / num_procesos
avg_respTime = sum(respTime) / num_procesos
cpu_utilization = ((total_time - idle_time) / total_time) * 100%
context_switches = num_cambios_de_proceso
```

Para Page Replacement:

```
faults = num_page_faults
hits = num_page_hits
hit_ratio = (hits / (hits + faults)) * 100%
total_references = length(reference_string)
```

5.6 Validación de Implementación

Casos de prueba realizados:

- Scheduling - FCFS:

    - Caso base: 4 procesos con burst times variados
    - Resultado: Proporcional a orden de llegada
- Scheduling - SJF:
    - Caso base: Procesos con burst times: 5,3,1,4
    - Resultado esperado: P3→P2→P4→P1
- Scheduling - RR (q=2):
    - Validación: Cada proceso ejecuta máximo quantum
    - Reinserción correcta en cola
- Scheduling - PRIORITY:
    - Validación: Orden según prioridad
- Page Replacement - FIFO:
    - Anomalía de Belady detectable
- Page Replacement - LRU:
    - Mejor que FIFO en la mayoría de escenarios
- Page Replacement - OPTIMAL:
    - Establece cota superior teórica

Manejo de concurrencia

El manejo de concurrencia en el sistema se refiere a la forma en que se ejecuta y controla dinámicamente la simulación de múltiples procesos a lo largo del tiempo. Este manejo se implementa mediante un mecanismo basado en ciclos repetitivos de ejecución, los cuales son generados a través de funciones temporizadas como setInterval, que actúan como un reloj lógico del sistema.

En cada ciclo de ejecución, el sistema realiza una serie de operaciones que permiten actualizar el estado global de la simulación. En primer lugar, se identifican los procesos que se encuentran en estado listo, los cuales residen en la cola de planificación. Posteriormente, se aplica el algoritmo de scheduling correspondiente para seleccionar el proceso que será ejecutado durante ese ciclo. Este proceso es entonces asignado al CPU, y su tiempo restante de ejecución se reduce en función del intervalo simulado.

De manera paralela, el sistema evalúa las condiciones que pueden provocar cambios en el estado de los procesos. Por ejemplo, si un proceso agota su tiempo de ejecución, es marcado como terminado y se libera el CPU. Si el sistema simula operaciones de entrada/salida, un proceso puede pasar al estado de espera, siendo temporalmente removido de la cola de listos. Estas transiciones son gestionadas mediante lógica condicional que se ejecuta en cada tick del reloj.

El manejo de concurrencia también implica la actualización continua de todas las entidades del sistema, incluso aquellas que no se encuentran en ejecución. Aunque un proceso no esté utilizando el CPU en un momento dado, su estado puede evolucionar en función del tiempo, lo que permite modelar de manera más realista la

coexistencia de múltiples procesos. Este comportamiento es esencial para simular sistemas de tiempo compartido, donde la percepción de simultaneidad se logra mediante la rápida alternancia entre procesos.

En el caso de la simulación multicore, este manejo se extiende para iterar sobre múltiples núcleos dentro de cada ciclo de ejecución. Cada core es actualizado de manera secuencial, pero independiente en términos lógicos, lo que permite simular la ejecución concurrente de múltiples procesos en diferentes unidades de procesamiento.

El manejo implementado es suficiente para reproducir el comportamiento observable de un sistema operativo en términos de planificación, ejecución intercalada y transición de estados.

El simulador traduce el diseño conceptual en una ejecución dinámica controlada por eventos temporales, permitiendo que múltiples procesos evolucionen de manera coherente dentro de un entorno simulado. Este enfoque logra representar de forma efectiva los principios operativos de la concurrencia, aun dentro de las limitaciones de un entorno de ejecución secuencial.

Resultados experimentales

Escenario 1: Análisis de Planificación de CPU (Scheduling)

Objetivo: Comparar el desempeño de los algoritmos Preemptive (Round Robin) vs. Non-Preemptive (FCFS) y el impacto del Quantum.

Configuración del Escenario:

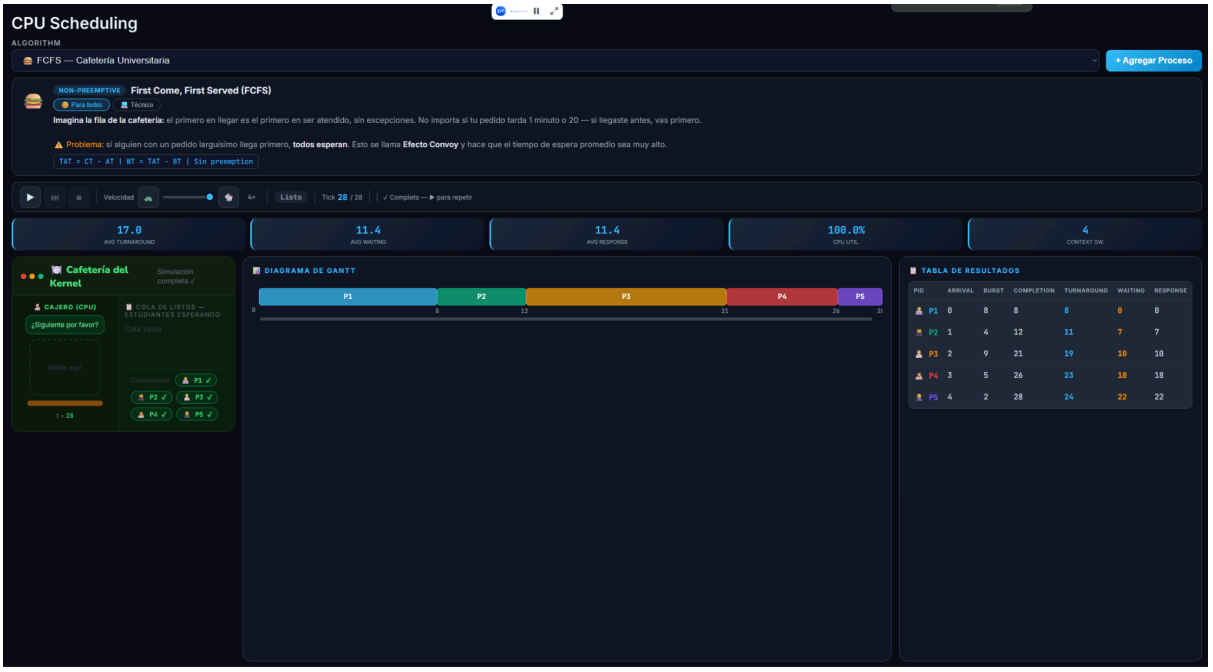
- **Procesos**: 5 procesos (P1 a P5).
- **Burst Times**: Variados (entre 5ms y 15ms).
- **Variable independiente**: Algoritmo (FCFS vs. RR con Quantum=4).

Datos de Entrada

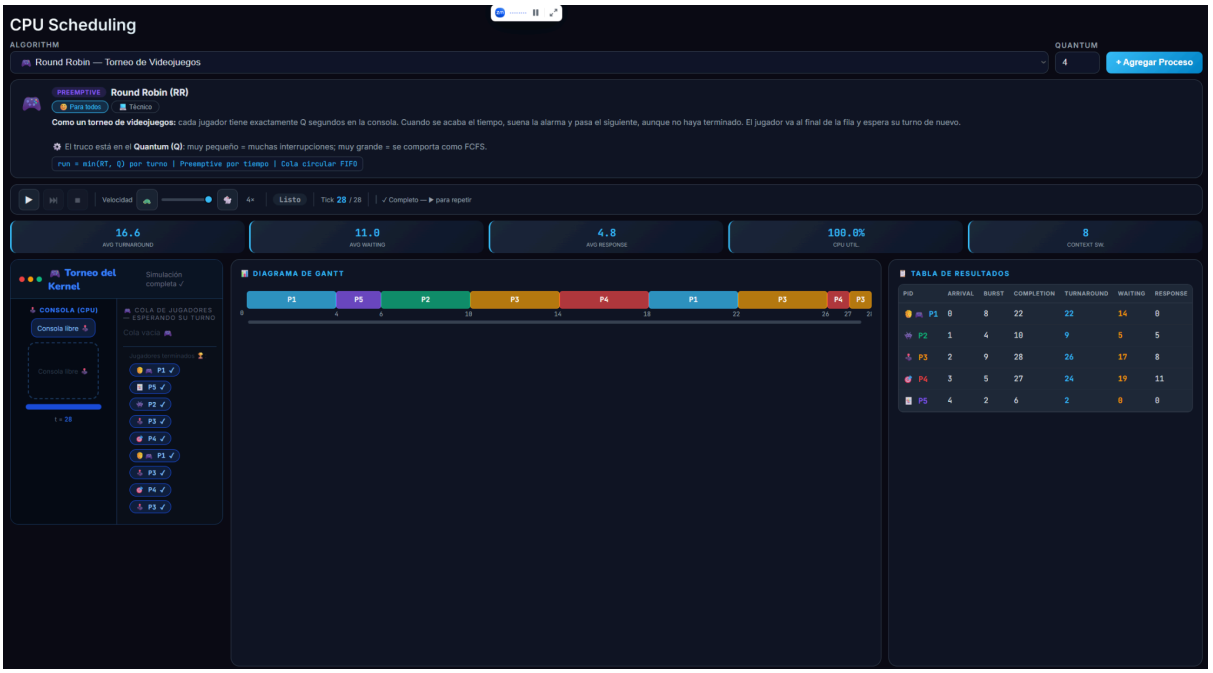
Proceso	Arrival Time (AT)	Burst Time (BT)
P1	0	**8**
P2	1	4
P3	2	9

P4	3	5
P5	4	2

Evidencia Visual - First Come First Serve



Evidencia Visual - Round Robin



Escenarios de prueba y evidencias

Análisis comparativo

Conclusiones

Trabajo futuro

IA

si se utiliza IA, será obligatorio incluir reflexión individual por integrante sobre qué aportó personalmente, qué verificó técnicamente y qué decisiones tomó el equipo para validar el contenido. De no integrarse esa reflexión individual de forma explícita y verificable, podrá aplicarse una sanción académica conforme a los lineamientos del curso.

Referencias

Brooks N. (2024). *Gestión de memoria en el sistema operativo: contigua, intercambio, fragmentación.* Guru99. Gestión de memoria en el sistema operativo: contigua, intercambio, fragmentación

Brooks N. (2024). *Memoria virtual en el sistema operativo: qué es, paginación bajo demanda, ventajas.* Guru99. Memoria virtual en el sistema operativo: qué es, paginación bajo demanda, ventajas

Brooks N. (2024). *Programación preventiva y no preventiva.* Guru99. Programación preventiva y no preventiva

Buzón, M. (2020). *Cuáles son los estados de los procesos de nuestros equipos.* Profesional Review. Cuáles son los estados de los procesos de nuestros equipos

Cortes D. (2024). *Introducción a los hilos (threads) y la concurrencia en programación.* Blender Deluxe. Introducción a los hilos (threads) y la concurrencia en programación

Cruzito R. (2025). *Asignación de memoria paginada: definición, propósito y estructura.* Estudiyando Asignación de memoria paginada: definición, propósito y estructura | Estudyando

Ti D. (2024). *Planificación de Procesos en Sistemas Operativos: Un Análisis Técnico Detallado.* Diego´s TI Munda. Planificación de Procesos en Sistemas Operativos: Un Análisis Técnico Detallado

Hu J. (2023). *Usa la función fork en C.* DelftStack. Usa la función fork en C | Delft Stack

Limited, H. G. (2025). *Gestión de procesos e hilos en sistemas operativos.* Hostragons. Gestión de procesos e hilos en sistemas operativos - Hostragons®

Martinez J. (2024). *Procesos: Definición y Funcionamiento en Sistemas Operativos.* SABERPUNTO. Procesos: Definición y Funcionamiento en Sistemas Operativos

TecnoDigital. (2023). *Gestión de procesos en sistemas operativos.* ITD. Gestión de procesos en sistemas operativos